TrustVault

Intelligent Secure Document Access & Audit System

Domain: Web Application Development / Cybersecurity

Project Type: Web-Based Secure Document Sharing Application

Technology Stack: Python, FastAPI, PostgreSQL, HTML, CSS, JavaScript

Table of Contents

1. Introduction
2. Project Overview
3. Problem Statement
4. Objectives
5. Scope of the Project
6. Technologies Used
7. System Architecture
8. Project Workflow
9. Project Modules and Implementation
10. Database Design
11. Security and Access Control
12. Audit Trail
13. Sample Output
14. Advantages
15. Limitations
16. Future Enhancements
17. Conclusion

1. Introduction

TrustVault is a secure document-sharing web application developed to provide controlled access to digital documents. The application focuses on protecting confidential files by allowing users to share documents with specific permissions.

In many document-sharing systems, users may lose control over shared files after granting access. TrustVault addresses this problem through user authentication, role-based access control, document-level permissions, and audit logging.

The application is developed using Python and FastAPI for backend operations, PostgreSQL for database management, and HTML, CSS, and JavaScript for the frontend.

TrustVault aims to provide a reliable environment where document owners can manage access permissions, control document activities, and maintain records of important actions.

2. Project Overview

TrustVault provides a web-based platform for managing and securely sharing documents.

The application includes user registration and login, document management, permission management, and audit log functionality.

Users can access documents based on the permissions granted to them. The system supports READ and EDIT permissions for individual documents. Document owners can manage permissions and revoke access when required.

The application also maintains audit records of important activities, helping administrators and authorized users review document-related actions.

The backend uses FastAPI to process requests and enforce access permissions. PostgreSQL stores user information, document metadata, permissions, and audit records.

Document file storage is designed to use a private Cloudflare R2 bucket.

3. Problem Statement

Sharing confidential documents through ordinary file-sharing methods can create security and privacy risks.

Once a document is shared, the owner may not have sufficient control over who can access it. Unauthorized users may attempt to view documents, and tracking document-related activities can become difficult.

The major problems include:

- Uncontrolled access to confidential documents.
- Difficulty managing individual user permissions.
- Lack of proper access revocation mechanisms.
- Limited visibility into document-related activities.
- Difficulty maintaining records of user actions.

TrustVault addresses these problems by providing authentication, document-level permission management, access control, and audit logging.

4. Objectives

The major objectives of TrustVault are:

- To develop a secure document-sharing web application.
- To implement user registration and authentication.
- To provide role-based access management.
- To implement document-level READ and EDIT permissions.
- To allow document owners to manage and revoke permissions.
- To prevent unauthorized document access through backend authorization checks.
- To maintain audit records of important application activities.
- To use PostgreSQL for structured data management.
- To provide a simple and user-friendly web interface.

5. Scope of the Project

TrustVault is designed for individuals and organizations that need controlled document sharing.

The application can be useful for:

- Organizations sharing confidential documents.
- Teams managing internal project files.
- Administrators controlling document access.
- Document owners sharing files with selected users.
- Users who need permission-based access to digital documents.

The current project focuses on authentication, role-based access, document permissions, document management, and audit logging.

6. Technologies Used

Programming Language

Python -- Used for backend development and application logic.

Backend Framework

FastAPI – Used to develop REST APIs and handle application requests.

Frontend Technologies

- HTML – Structures the web pages.
- CSS – Provides styling and layout.
- JavaScript – Handles frontend interactions.

Database

PostgreSQL – Stores user accounts, document metadata, access permissions, and audit records.

Authentication and Security

- JWT – Used for token-based authentication.
- Argon2 – Used for password hashing.
- Role-Based Access Control – Controls access based on user roles.
- Document-Level Permissions – Supports READ and EDIT access.

Cloud Storage

Cloudflare R2 – Designed for private storage of document files.

Development Tools

- Visual Studio Code
- Git
- GitHub
- PostgreSQL tools

- Render – Used for application deployment.

7. System Architecture

The TrustVault application follows a client-server architecture.

The frontend communicates with the backend through HTTP requests. FastAPI processes the requests, verifies authentication, checks permissions, and performs the required operations.

PostgreSQL stores structured application data, including user information, document metadata, permission records, and audit logs.

Document file contents are designed to be stored in a private Cloudflare R2 bucket.

Main Components

1. Frontend Module:
   Provides the user interface for registration, login, document management, and other application operations.
2. Authentication Module:
   Verifies user credentials and manages JWT-based authentication.
3. Authorization Module:
   Checks user roles and document-level permissions before allowing protected operations.
4. Document Management Module:
   Handles document metadata, upload, search, download, replacement, and owner-only deletion operations.
5. Database Module:
   Manages application data using PostgreSQL.
6. Audit Module:
   Records important application activities and supports hash-chain verification of audit records.

8. Project Workflow

The workflow of the project is as follows:

START

↓

Open TrustVault Application

↓

User Registration / Login

↓

Validate User Credentials

↓

Generate Authentication Token

↓

Access User Dashboard

↓

Select Document Operation

↓

Upload / View / Search / Manage Documents

↓

Check User Authentication and Permissions

↓

Allow Authorized Operation

↓

Record Important Activity in Audit Log

↓

Return Response to User

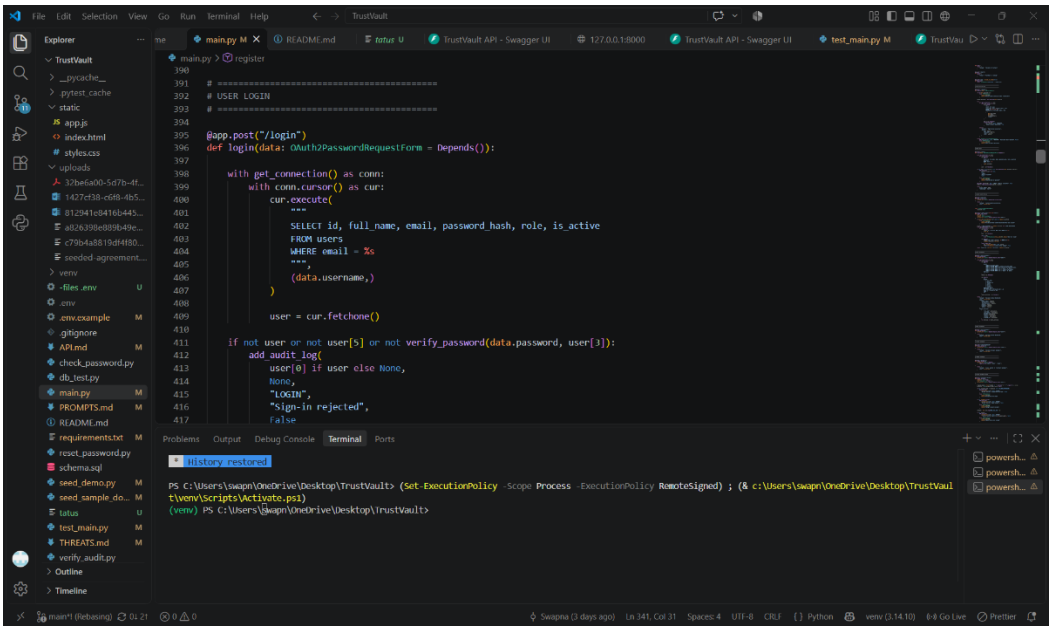
↓

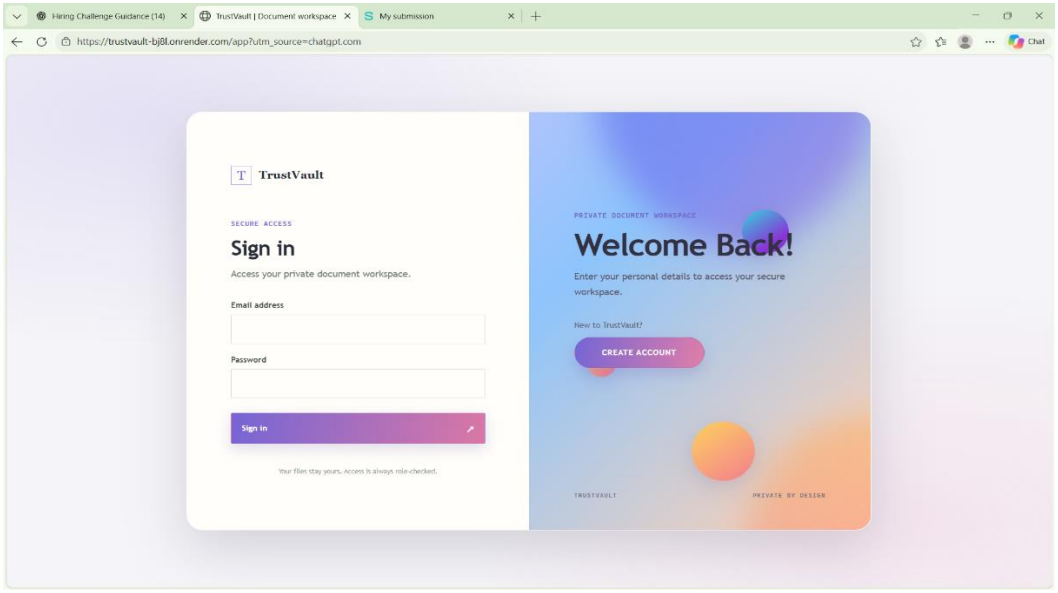
Logout / Exit Application

END

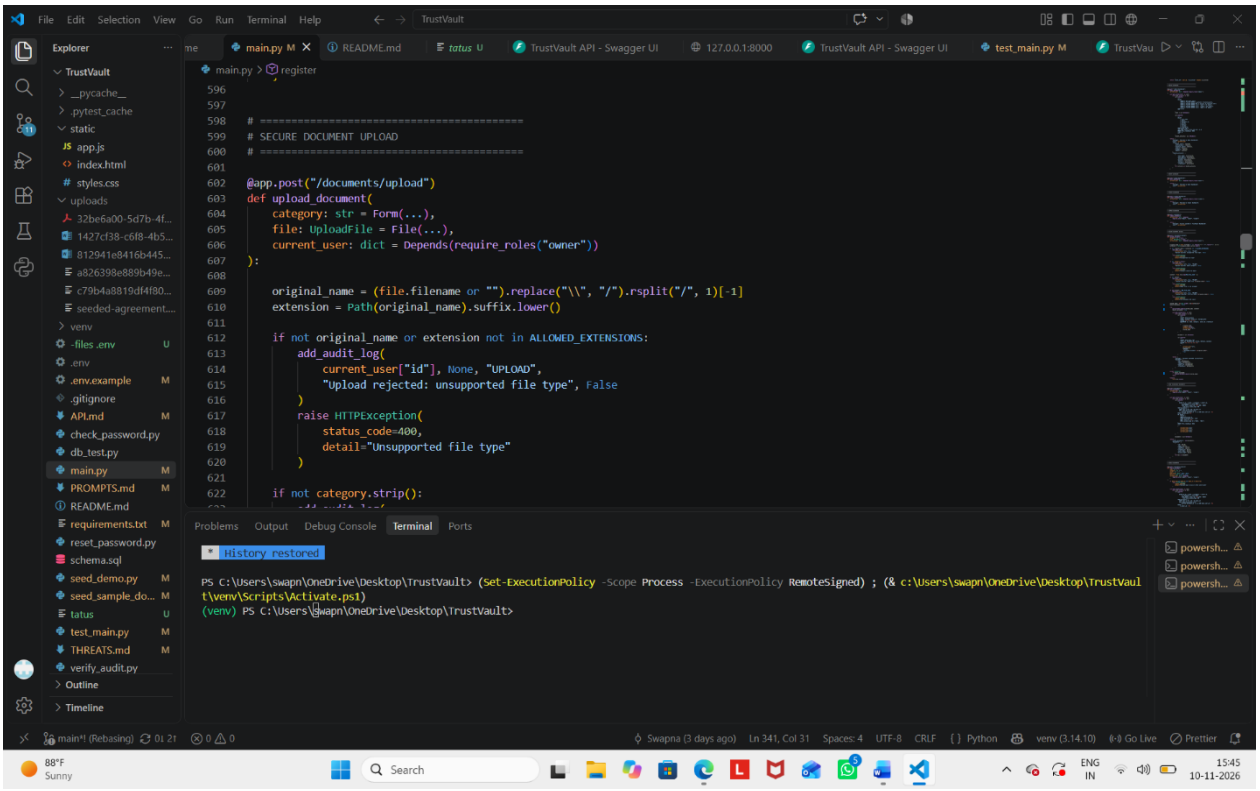
9. Program Modules

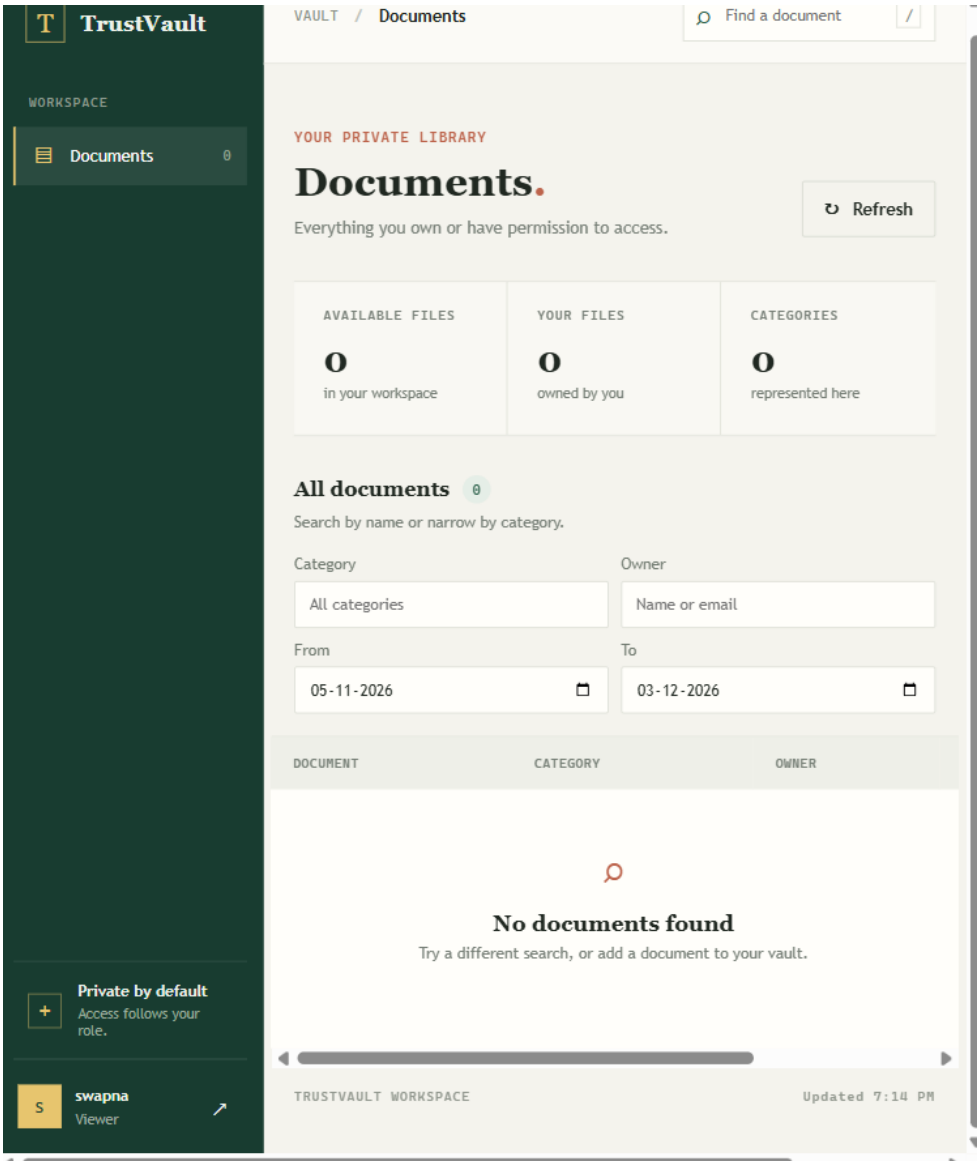
Module 9.1 user authentication module



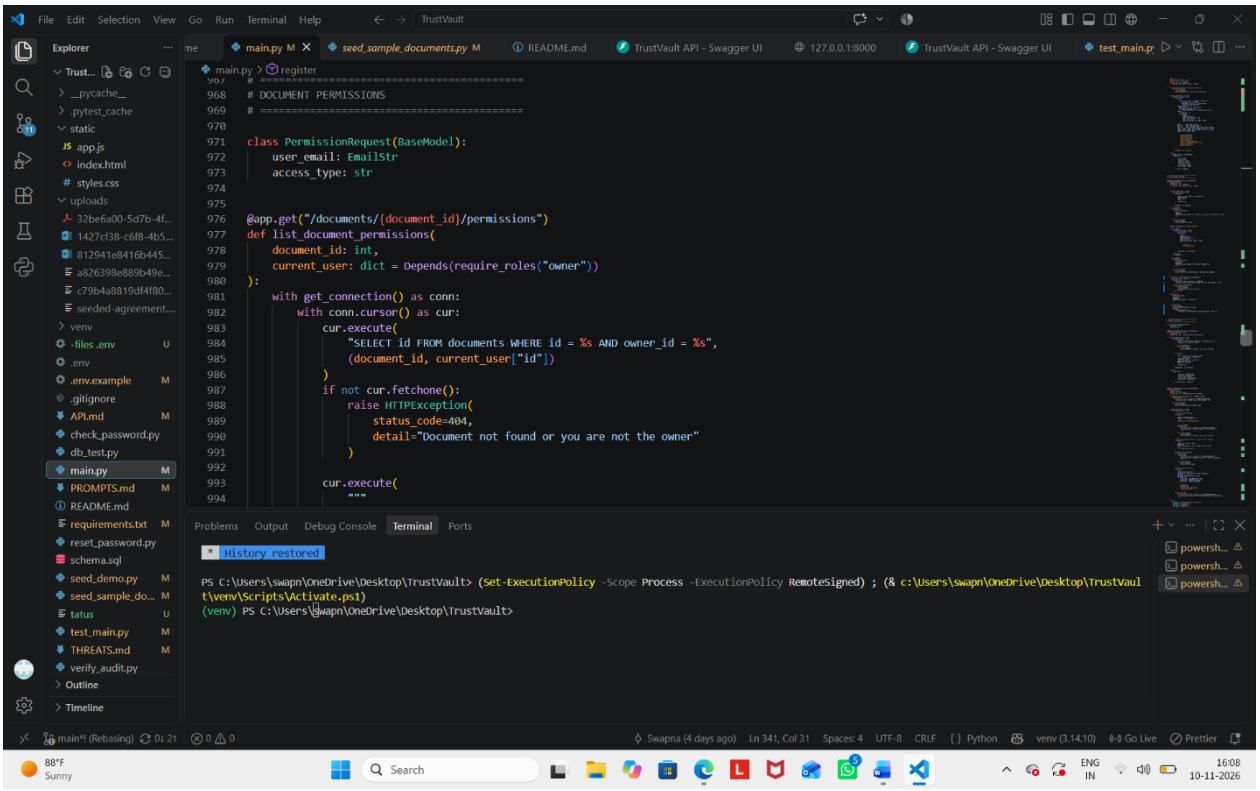


9.2 Document Management Module


File Edit Selection View Go Run Terminal Help
TrustVault
Explorer
main.py M
README.md
tatus U
TrustVault API - Swagger UI
127.0.0.1:8000
TrustVault API - Swagger UI
test_main.py M
main.py > register
==
SECURE DOCUMENT UPLOAD
==
@app.post("/documents/upload")
def upload_document(
category: str = Form(...),
file: UploadFile = File(...),
current_user: dict = Depends(require_roles("owner"))
):
original_name = (file.filename or "").replace("\\", "/").rsplit("/", 1)[-1]
extension = Path(original_name).suffix.lower()
if not original_name or extension not in ALLOWED_EXTENSIONS:
add_audit_log(
current_user["id"], None, "UPLOAD",
"Upload rejected: unsupported file type", False
)
raise HTTPException(
status_code=400,
detail="Unsupported file type"
)
if not category.strip():
Problems Output Debug Console Terminal Ports
History restored
PS C:\Users\swapn\OneDrive\Desktop\TrustVault> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\swapn\OneDrive\Desktop\TrustVault\venv\Scripts\Activate.ps1)
(venv) PS C:\Users\swapn\OneDrive\Desktop\TrustVault>
88°F
Sunny
Search
ENG
IN
15:45
10-11-2026



9.3 Permission Management Module



10. Database design

TrustVault uses PostgreSQL as its relational database.
The database manages important application information.

Main Database Entities

Users:
Stores user account details, email addresses, password hashes, roles, and account status.

Documents:
Stores document metadata and storage references.

Permissions:
Stores document-level access permissions assigned to users.

Audit Logs:
Stores records of important application activities and related audit information.

Database Operations

The application uses PostgreSQL to perform operations such as:
Creating user accounts.
Retrieving user information.
Managing document metadata.
Granting and revoking permissions.
Recording audit events.
Retrieving audit records.

Document file contents are intended to be stored separately in a private Cloudflare R2 bucket, while PostgreSQL maintains their metadata and storage references.

11. Security access control

Security is one of the primary objectives of TrustVault.

The application uses multiple security mechanisms to protect document access.

Password Security

Passwords are hashed using Argon2 before being stored in the database. Plain-text passwords are not intended to be stored.

JWT Authentication

JWT tokens are used to authenticate users when accessing protected application endpoints.

Role-Based Access Control

The application distinguishes between Admin, Owner, and Viewer roles.

Document-Level Authorization

The backend checks the permissions associated with a document before allowing protected operations.

Permission Revocation

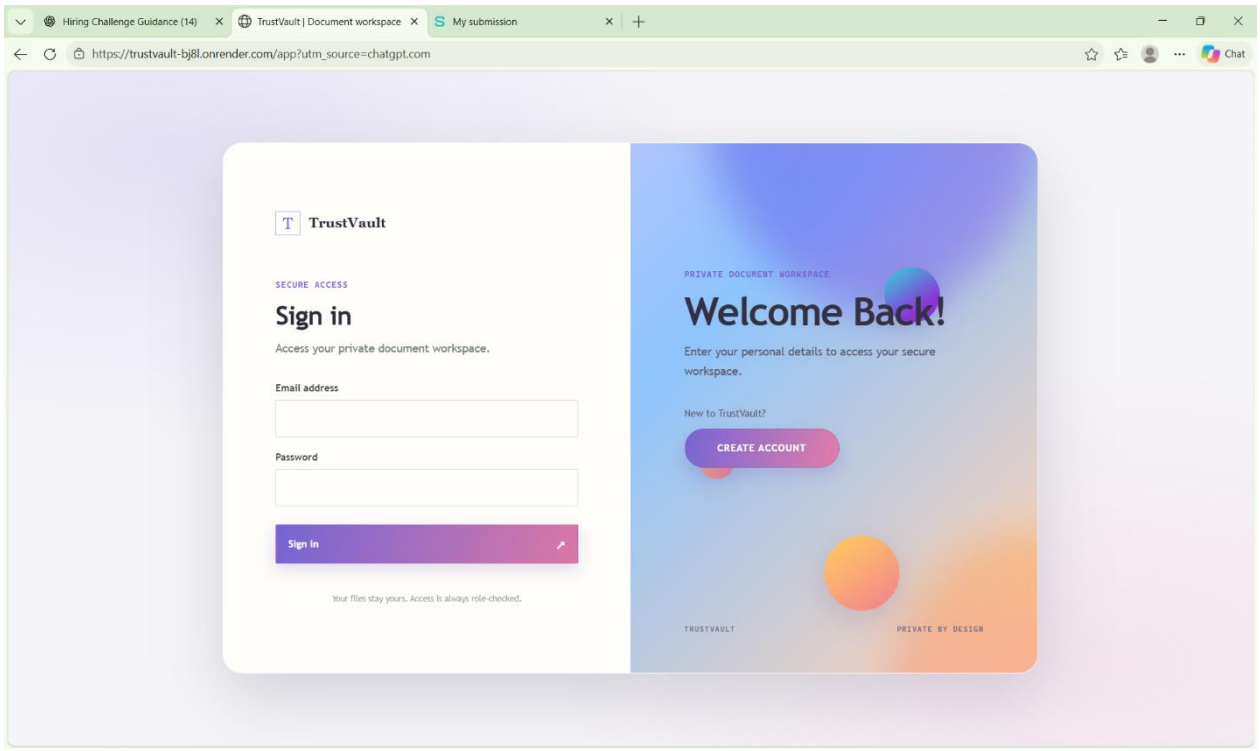
Document owners can revoke previously granted permissions to prevent further authorized access.

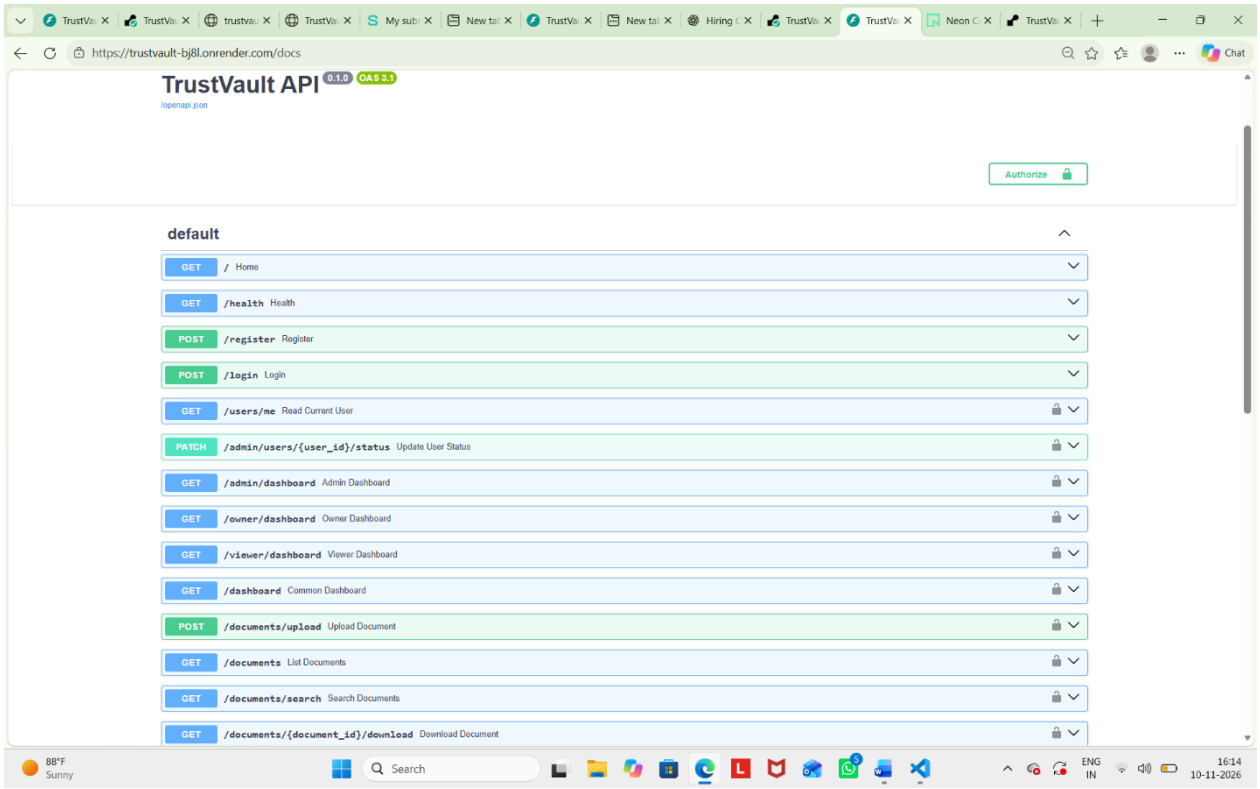
The frontend interface alone does not provide security. Access decisions are enforced by the backend.

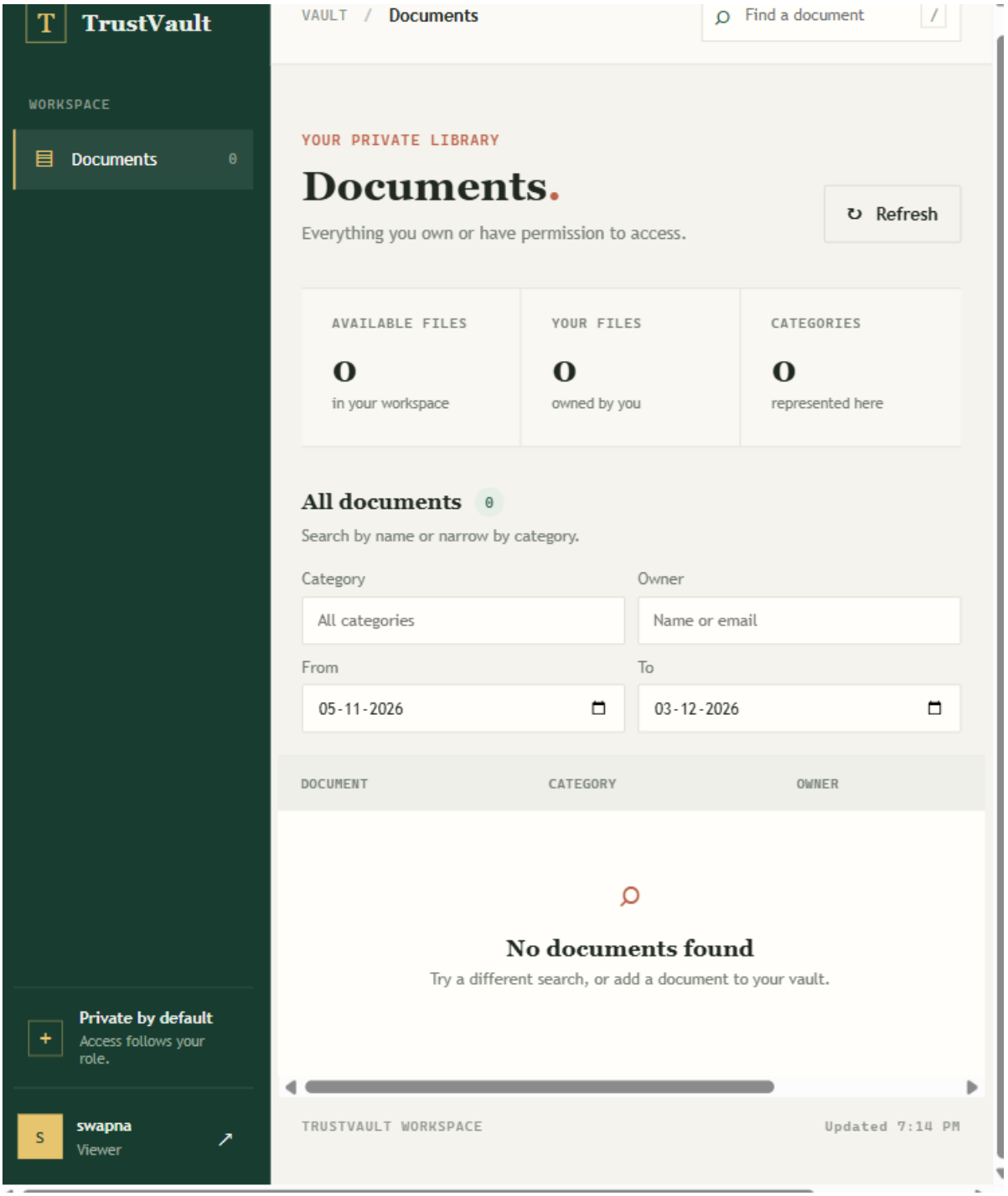
12. Audit Trail

An audit trail is a record of important activities performed within an application.
TrustVault maintains audit records to help track document-related operations and user activities.
The audit system uses a hash-chain mechanism and provides a verification helper to check the integrity of recorded audit entries.
The audit functionality helps improve transparency and accountability in document sharing.

13. Sample outputs







14.Advantages

- Provides controlled document sharing.
- Supports user authentication.
- Implements role-based access control.
- Provides individual document permissions.
- Supports permission revocation.

- Maintains audit records of important activities.
- Uses PostgreSQL for structured data management.
- Provides a web-based interface.
- Uses a modular backend architecture.

15.Limitations

- The current implementation has certain limitations:
- Multi-factor authentication (MFA) is not built in.
- Login rate limiting requires further implementation.
- Malware scanning is not currently included.
- Advanced file-content validation needs improvement.
- Automated backup and disaster recovery mechanisms require further development.
- Document versioning is not currently available.
- An independent external anchor for the audit hash chain is not implemented.
- Real Cloudflare R2 credentials and bucket configuration must be verified to confirm cloud document-storage operations.

16.Future Enhancements

The following features can be implemented in future versions:

- Multi-factor authentication for stronger account security.
- Login rate limiting and suspicious activity detection.
- Malware scanning for uploaded documents.
- Advanced file validation.
- Document version management.
- Automated backup and recovery.
- Improved audit monitoring and external integrity verification.
- Email notifications for document-sharing and permission changes.
- Enhanced dashboard and reporting features.

17.Conclusion

AI tools were used as development-support tools during the development of TrustVault.

ChatGPT was used to:

- Understand technical concepts related to FastAPI and Python.

- Troubleshoot programming errors.
- Understand authentication and authorization implementation.
- Review code and improve development workflows.
- Organize project documentation.
- Understand database integration and deployment-related issues.

AI-generated suggestions were reviewed and used as guidance during development. The application was tested during implementation, and final project decisions were made by the developer.

AI assistance is disclosed transparently as part of the development process.